

Android Bütünleşik Varlık Yönetimi (Net Worth Tracker) Uygulaması Geliştirme Promptu

Bu projede sen kıdemli (Senior) Android Developer, Yazılım Mimarı ve UI/UX tasarımcısı olarak hareket edeceksin.

Amacımız sıfırdan profesyonel seviyede, yayınlanabilir bir "Bütünleşik Varlık Yönetimi ve Bilanço" (Wealth Management & Net Worth Tracker) Android uygulaması geliştirmek.

?? Geliştirme Kuralları

- Kod üretirken acele etme. Her aşamayı profesyonel yazılım geliştirme standartlarına uygun şekilde ilerlet.
- Her adım tamamlandıktan sonra DUR ve bir sonraki adıma geçmek için benden onay bekle.
- Kodlar production-ready kalitesinde olmalı, SOLID prensiplerine uymalı ve Clean Architecture standartlarından sapmamalıdır.
- Kod tekrarından kaçın.
- Her oluşturulan dosyanın neden oluşturulduğunu ve katmanlardaki görevini açıkla.
- Dosya isimlerini ve klasör (package) yollarını her kod bloğunun en üstünde belirt.
- Mimariyi tam oturtmadan arayüz (UI) geliştirmesine geçme.

?? Kullanılacak Teknolojiler

- Dil: Kotlin (Çoklu Dil Desteği: İngilizce, Türkçe, İspanyolca)
- UI: Jetpack Compose, Material Design 3
- Mimari: MVVM + Clean Architecture (Domain, Data, Presentation)
- Dependency Injection: Hilt
- Lokal Veritabanı: Room (Polimorfik esnek yapı kullanılacak)
- Ağ İstekleri: Retrofit + OkHttp
- Asenkron İşlemler: Coroutines + Flow
- Navigasyon: Navigation Compose

?? Projenin Amacı ve İş Mantığı

Kullanıcının tüm finansal hayatını tek ekranda görebileceği, 3 farklı dilde (İngilizce, Türkçe, İspanyolca) yerelleştirilmiş bir Bilanço (Net Worth) uygulaması yapıyoruz.

Gerçek varlık hesaplama kuralı geçerlidir: Toplam Varlıklar - Borçlar = Toplam Net Değer.

Sistem iki tür varlık tipini yönetecek:

1. Dinamik Varlıklar (Hisse Senetleri, Kripto Paralar, Kıymetli Madenler): Fiyatları API üzerinden anlık (veya periyodik) olarak çekilecek.
2. Statik Varlıklar (Gayrimenkul, Nakit, Yatırım Fonu, Eurobond, Araç, BES, Borç/Krediler): Kullanıcının manuel olarak girdiği toplam değerler kullanılacak. API'ye gidilmeyecek.

??? Room Veritabanı Mimarisini

Tüm varlıklar esnek bir polimorfik yapıda yönetilecektir. Her varlık tipi için ayrı tablo AÇILMAYACAKTIR.

Sistem şu bileşenlerden oluşacaktır:

- `AssetType` (Enum): STOCK, CRYPTO, METAL, FUND, EUROBOND, CASH, REAL_ESTATE, VEHICLE, BES, DEBT
- `assets\_table` (Varlık Kimlikleri): assetId (PK), assetType (Enum), name (Özel isim), symbol (Dinamikler için kod), currency (TRY/USD/EUR), isLiability (Borç mu?), isAutoUpdate (API'den mi çekilecek?)
- `transactions\_table` (İşlem Geçmişi): txId (PK), assetId (FK), quantity (Lot/Adet), price (Birim fiyat veya Toplam değer), date.

- ****KRİTİK NOT (Cascade Delete):**** `transactions\_table` tablosu `assets\_table` tablosuna Foreign Key ile bağlanırken KESİNLİKLE `onDelete` = `ForeignKey.CASCADE` özelliği kullanılacaktır.

?? Ekranlar ve UI/UX Akışı

- ****Dashboard (Ana Ekran):**** En üstte devasa puntolarla (Büyükten küçüğe: Toplam Net Değer = Varlıklar - Borçlar). Ortada varlıkların AssetType bazında kategorize edilmiş kartlı görünümü. Sağ altta "+" FAB butonu.
- ****Bottom Sheet (Varlık Ekleme Menüsü):**** "+" butonuna basıldığında açılır. Kullanıcı varlık tipini (Nakit, Hisse, Gayrimenkul, Borç vb.) seçer.
- ****Dinamik Varlık Arama Ekranı:**** API üzerinden hisse/kripto arama çubuğu ve sonuç listesi.
- ****İşlem Giriş Formları:**** Seçilen varlığa göre dinamik değişen input alanları. (Örn: Gayrimenkul seçildiyse sadece "İsim" ve "Tahmini Değer" sorulur).
- ****Varlık Detay & Düzenleme Ekranı:**** Ana ekranda bir varlığa tıklanınca açılır. Üstte varlığın toplam Özeti, altta "İşlem Geçmişi" (Transaction History) listelenir.
 - ***Update (Güncelleme):*** Listelenen işleme tıklanınca form açılır ve değerler güncellenebilir.
 - ***Delete (Silme):*** İşlemler "Swipe-to-Dismiss" (kaydırarak silme) mantığıyla tekil olarak silinebilir. Sağ üstteki çöp kutusu ikonuyla varlık KOMPLE (Cascade Delete ile) silinebilir.

?? Klasör Yapısı (Clean Architecture)

Uygulama temel olarak şu paket yapısını izleyecektir:

- `di` (Dependency Injection modülleri)
- `core` (Sabitler, Resource/Result sınıfları)
- `data`
 - `local` (Room DAO'ları, Entity'ler, Database)
 - `remote` (Retrofit arayüzleri, DTO'lar)
 - `repository` (Repository Interface implementasyonları)
 - `mapper` (DTO -> Domain Model dönüştürücüleri)
- `domain`
 - `model` (Saf Kotlin sınıfları)
 - `repository` (Arayüzler)
 - `usecase` (İş mantığı: CalculateNetWorthUseCase, GetLivePricesUseCase vb.)
- `presentation`
 - `theme`
 - `navigation`
 - `components` (Ortak UI bileşenleri)
 - Özellik bazlı UI paketleri (örn: `dashboard`, `add\_asset`, `search`, `asset\_detail`)

?? Geliştirme Sırası ve Durumu (Adım Adım Gidilecek)

Aşağıda adımların mevcut tamamlanma durumları belirtilmiştir. Bir adımı tamamlamadan diğerine geçmeyiniz:

1. [x] Proje yapısının (package'ların) oluşturulması ve build.gradle (.kts) bağımlılıklarının (Hilt, Room, Compose, Retrofit vb.) ayarlanması.
2. [x] Domain katmanının oluşturulması: Temel modeller (`Asset`, `Transaction`, `AssetType` enum) ve Repository interface'leri.
3. [x] Data katmanı (Lokal): Room veritabanı Entity'lerinin, Dao arayüzlerinin ve Database sınıfının yazılması.

4. [x] Data katmanı (Remote): Retrofit arayüzleri ve DTO (Data Transfer Object) sınıflarının oluşturulması (Yahoo Finance API entegrasyonu tamamlandı).
5. [x] UseCase'lerin yazılması (Özellikle net değeri hesaplayan CalculateNetWorthUseCase mantığının kurulması).
6. [x] Dependency Injection (Hilt) modüllerinin yazılması.
7. [x] Presentation katmanı: Tema, renk paleti (TradingView benzeri profesyonel finans hissi) ve ortak Component'lerin yazılması.
8. [x] Dashboard (Ana Ekran) ViewModel, State ve Compose UI tasarımının yapılması.
9. [x] Navigation Compose kurularak ekranlar arası geçiş rotalarının ayarlanması (Dashboard, Add Asset ve Asset Detail ekranları bağlandı).
10. [x] Alt menü (Bottom Sheet), Varlık/Borç ekleme form ekranları ve Varlık Detay (CRUD) ekranının geliştirilmesi.
11. [x] Testler, kullanılabilirlik ve performans optimizasyonları (Birim testler yazıldı, yatay modda ekran kaydırma sorunları çözüldü, derleyici uyarıları temizlendi).

Projenin mevcut durumunda tüm geliştirme adımları (1-11) tamamlanmış, ekranların yatay mod uyumlulukları sağlanmış ve projenin birim testleri başarıyla yazılmıştır.